

ENTREGA GRAFOS: PROBLEMA 4



UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE CIENCIAS MATEMÁTICAS
MÁSTER EN INGENIERÍA MATEMÁTICA

PROFESOR: José María Ferrer Caja

ALUMNA: Siria Catherine Íñiguez Brito

ASIGNATURA: Modelos Determinísticos en Logística

CURSO: 2025-2026

Madrid, 17 de Noviembre de 2025

Índice

1. INTRODUCCIÓN	2
2. ALGORITMO DE DIJKSTRA	3
2.1. Fases del Procedimiento Iterativo para Dijkstra	3
2.2. Implementación del Algoritmo de Dijkstra en Python	4
3. ALGORITMO DE PRIM	9
3.1. Fases del Procedimiento Iterativo para Prim	9
3.2. Implementación del Algoritmo de Prim en Python	10
4. APLICACIÓN AL PROBLEMA DE FUENFRÍA	13
4.1. Parte a) Cálculo de Caminos Más Cortos	13
4.1.1. Análisis de la Solución Obtenida	14
4.2. Parte b) Minimizar Kilómetros de Línea Tendida	15
4.2.1. Análisis de la Solución Obtenida	16

1. INTRODUCCIÓN

El presente informe tiene como objetivo principal la aplicación práctica de dos de los algoritmos fundamentales y más utilizados en la **teoría de grafos** y la optimización de redes: el algoritmo de **Dijkstra** y el algoritmo de **Prim**. El algoritmo de Dijkstra se empleará para calcular los **caminos más cortos** desde un nodo fuente a todos los demás, resultando esencial en problemas de enrutamiento y logística. Por su parte, el algoritmo de Prim será utilizado para determinar el **árbol de expansión mínima (AEM)**, una estructura crucial para la conectividad eficiente de todos los nodos con el mínimo coste total.

Estos algoritmos se aplican para resolver el Problema 4 de la sección VI (Optimización en redes), planteado en los apuntes de la asignatura Modelos Determinísticos en Logística. Este ejercicio aborda la optimización de rutas y el tendido de líneas en el Parque de la Fuenfría. Presenta un escenario donde la correcta elección del algoritmo —Prim para la conectividad mínima y Dijkstra para la eficiencia de las rutas— es determinante para minimizar los costes operacionales y la distancia recorrida. A lo largo del informe, se detallará la modelización del problema en un grafo, la implementación de los códigos en Python y la interpretación de los resultados obtenidos.

2. ALGORITMO DE DIJKSTRA

El **Algoritmo de Dijkstra** es un método fundamental en la optimización de redes, diseñado para resolver el problema del camino más corto desde un nodo fuente a todos los demás en un grafo dirigido con pesos de arista no negativos.

- ✓ Su requisito principal es que todas las distancias sean estrictamente **no negativas**.
- ✓ El algoritmo exhibe una complejidad de tipo polinomial, haciéndolo eficiente para la mayoría de las aplicaciones prácticas.
- ✓ Su mecánica se basa en el mantenimiento de dos conjuntos de nodos: P para nodos **permanentes** (distancia final determinada) y T para nodos **transitorios** (distancia aún sujeta a mejora).
- ✓ La reconstrucción del camino óptimo se logra mediante el registro de $pred(j)$, que es el nodo predecesor de j en el camino más corto desde el origen.

2.1. Fases del Procedimiento Iterativo para Dijkstra

PASO 0. INICIALIZACIÓN

$$u_1 = 0, u_j = d_{1j}, j = 2, \dots, n. P = \{1\}, T = \{2, \dots, n\}, pred(j) = 1, j = 2, \dots, n$$

PASO 1. HACER PERMANENTE EL NODO CON DISTANCIA MÍNIMA

1. Elegir un nodo k del conjunto **Transitorio** (T) tal que su distancia acumulada (u_k) sea la mínima:

$$u_k = \min_{j \in T} \{u_j\}$$

2. Mover el nodo k al conjunto **Permanente** (P):

$$P \leftarrow P \cup \{k\} \quad \text{y} \quad T \leftarrow T - \{k\}$$

3. **Condición de Parada:** Si el conjunto de nodos transitorios (T) está vacío, **PARAR** el algoritmo.

PASO 2. REVISIÓN Y ACTUALIZACIÓN DE NODOS TRANSITORIOS

1. Para cada nodo j en el conjunto **Transitorio** (T), calcular la nueva distancia a través del nodo k recién hecho permanente.
2. Actualizar la distancia de j (u_j) eligiendo el camino más corto:

$$u_j \leftarrow \min\{u_j, u_k + d_{kj}\}$$

3. Si se modifica la distancia u_j , actualizar el **predecesor** de j :

$$pred(j) \leftarrow k$$

4. Volver al **PASO 1**.

2.2. Implementación del Algoritmo de Dijkstra en Python

Para la implementación del Algoritmo de Dijkstra se ha utilizado Python como lenguaje de programación. El código, contenido en el archivo `Algoritmo_Dijkstra.py`, sigue una aproximación tradicional basada en la gestión de una **matriz de adyacencia** y el etiquetado explícito de los nodos como Permanentes ('P') o Transitorios ('T').

El código se organiza en cuatro bloques principales para facilitar su comprensión: la construcción del grafo, las funciones auxiliares, el algoritmo principal (con sus funciones de interfaz) y la aplicación a unos datos concretos.

```
1
2 INF = float('inf')
3
4 # -----
5 # BLOQUE 1: REPRESENTACION DEL GRAFO POR MATRIZ DE ADYACENCIA
6 # -----
7
8 def aristas_a_matriz(aristas: dict, nodos: list):
9
10     """
11     Convierte la representacion del grafo de aristas (diccionario) a
12     una matriz de adyacencia asumiendo un grafo dirigido.
13     """
14
15     num_nodos = len(nodos)
16
17     # 1. Inicializar la matriz con 0 en la diagonal e INF en el resto
18     matriz_adyacencia = []
19
20     for i in range (0,num_nodos):
21         fila = []
22         for j in range (0,num_nodos):
23             if (i==j):
24                 fila.append(0.0) # Distancia de un nodo a si mismo es 0
25             else:
26                 fila.append(INF) # No hay conexion directa
27
28         matriz_adyacencia.append(fila)
29
30     # 2. Mapear nombres de nodos a indices (0, 1, 2, ...)
31     mapeo_nodos = {nodo: i for i, nodo in enumerate(nodos)}
32
33     # 3. Rellenar la matriz con los pesos de las aristas
34     for arista, peso in aristas.items():
35
36         # Las aristas se esperan en formato 'Origen-Destino'
37         salida, llegada = arista.split('-')
38
39         i = mapeo_nodos[salida]
40         j = mapeo_nodos[llegada]
41
42         matriz_adyacencia[i][j] = float(peso)
43
44     return matriz_adyacencia
```

```

45
46 # -----
47 # BLOQUE 2: FUNCIONES AUXILIARES
48 # -----
49
50 def Buscar_minimo (vector_u, lista_estados):
51
52     """
53     Busca el nodo con la menor distancia entre aquellos que aun estan
54     en estado Temporal ('T').
55     """
56
57     minimo = INF
58     minimo_posicion = -1
59
60     # Recorrer todas las distancias
61     for i in range (0, len(vector_u)):
62         if lista_estados[i] == 'T':
63             candidato = vector_u[i]
64
65             # Comprobar si el candidato es menor que el minimo actual y
66             es accesible (no INF)
67             if candidato < minimo and candidato != INF:
68                 minimo = candidato
69                 minimo_posicion = i # Guardar el indice (posicion) del
70                 nodo minimo
71
72     encontrado = (minimo_posicion != -1)
73
74     return encontrado, minimo_posicion
75
76 def Actualizar_vectores(vector_u, lista_estados, lista_predecesor,
77 matriz, minimo_posicion):
78
79     """
80     Realiza las operaciones de actualizacion.
81     """
82
83     for j in range (0, len(vector_u)):
84         # Itera sobre todos los nodos del grafo. 'j' representa el nodo que
85         estamos revisando.
86
87         if (lista_estados[j] == 'T'):
88             # Condicion 1: Solo revisamos los nodos que aun estan en T.
89
90             nueva_distancia = vector_u[minimo_posicion] + matriz[
91             minimo_posicion][j]
92
93             if nueva_distancia < vector_u[j]:
94                 # Condicion 2: Comprueba si el nuevo camino es mas corto
95                 que el camino mas corto conocido actualmente hacia el nodo 'j'
96                 vector_u[j] = nueva_distancia
97                 lista_predecesor[j] = minimo_posicion
98
99     return vector_u, lista_estados, lista_predecesor

```

```

96
97 # -----
98 # BLOQUE 3: ALGORITMO PRINCIPAL Y FUNCIONES DE EJECUCION
99 # -----
100
101 def Algoritmo_Dijkstra(aristas: dict, nodos: list, nodo_raiz):
102
103     """
104     Implementacion principal del Algoritmo de Dijkstra basado en matriz
105     .
106     """
107
108     num_nodos = len(nodos)
109     mapeo_nodos = {nodo: i for i, nodo in enumerate(nodos)}
110
111     # 1. Validacion de la raiz
112     if nodo_raiz not in mapeo_nodos:
113         return "ERROR: Nodo raiz no encontrado.", None, None
114
115     # 2. Conversion a matriz y obtencion de distancias iniciales
116     matriz_adyacencia = aristas_a_matriz(aristas, nodos)
117
118     # El vector_u se inicializa con la fila de la matriz del nodo raiz
119     vector_u = matriz_adyacencia[mapeo_nodos[nodo_raiz]].copy()
120
121     # 3. Inicializacion de estados y predecesores
122     lista_estados = ['T']* num_nodos
123     lista_estados[mapeo_nodos[nodo_raiz]] = 'P'
124     lista_predecesor = [mapeo_nodos[nodo_raiz]]* num_nodos
125
126     m = 1 # Contador de nodos etiquetados permanentemente
127
128     # 4. Bucle principal de Dijkstra
129     while m < num_nodos :
130         # a. Buscar el nodo temporal mas cercano
131         encontrado, minimo_posicion = Buscar_minimo(vector_u,
132             lista_estados)
133
134         # b. Si no se encuentra un nodo temporal accesible, el grafo
135         # esta desconectado
136         if not encontrado and m < num_nodos:
137             break
138
139         # c. Etiquetar el nodo encontrado como Permanente ('P')
140         lista_estados[minimo_posicion] = 'P'
141
142         m += 1
143
144         # d. Relajar aristas salientes del nodo permanente
145         Actualizar_vectores(vector_u, lista_estados, lista_predecesor,
146             matriz_adyacencia, minimo_posicion)
147
148     # 5. Devolver resultados
149     if m < num_nodos:
150         return "INFECTIBLE: El grafo esta desconectado.", None, None
151
152     return "EXITO", vector_u, lista_predecesor

```

```

150
151 def Reconstruir_Grafo_Orientado(nodos: list, lista_predecesor: list,
152     vector_u: list):
153     """
154     Reconstruye el conjunto de aristas que forman el Arbol de Caminos
155     Mas Cortos.
156     """
157     aristas_grafo = []
158     mapeo_inverso = {i: nodo for i, nodo in enumerate(nodos)}
159
160     for i, predecesor_index in enumerate(lista_predecesor):
161         if vector_u[i] == INF or vector_u[i] == 0.0:
162             continue
163
164         destino = mapeo_inverso[i]
165         origen = mapeo_inverso[predecesor_index]
166
167         arista_grafo = f"{origen}-{destino}"
168         aristas_grafo.append(arista_grafo)
169
170     return list(set(aristas_grafo))
171
172
173 def Algoritmo_Dijkstra_final(aristas: dict, nodos: list, nodo_raiz):
174     """
175     Funcion de interfaz: ejecuta Dijkstra, maneja errores y formatea la
176     salida.
177     """
178
179     # 1. Ejecutar Dijkstra
180     estado, vector_u, lista_predecesor = Algoritmo_Dijkstra(aristas,
181         nodos, nodo_raiz)
182
183     # 2. Manejar Errores
184     if estado.startswith("ERROR"):
185         return {"Estado": estado, "Grafo_Orientado": None, "Distancias":
186             None}
187
188     # 3. Reconstruir el Grafo
189     grafo_orientado_final = Reconstruir_Grafo_Orientado(nodos,
190         lista_predecesor, vector_u)
191
192     # 4. Formatear la salida de distancias
193     distancias_finales = {nodos[i]: round(d, 2) for i, d in enumerate(
194         vector_u)}
195
196     # 5. Devolver el resultado
197     return {"Estado": "Exito Total" if estado == "EXITO" else "Exito
198         Parcial/Infactible", "Grafo_Orientado": grafo_orientado_final, "
199         Distancias": distancias_finales, "Detalle": "El grafo final contiene
200         solo las aristas que forman los caminos mas cortos desde la raiz."
201     }

```

Listing 1: Código Python del Algoritmo de Dijkstra (Algoritmo_Dijkstra.py)

Para la resolución del caso práctico, cuyos detalles se abordan en la Sección 4.1, el siguiente bloque de código especifica los datos de entrada (aristas, nodos y nodo raíz) y ejecuta la función `Algoritmo_Dijkstra_final`. La salida de esta ejecución, que incluye el grafo orientado de caminos más cortos y las distancias finales, será analizada en la sección 4.1.1.

```
1
2 # -----
3 # BLOQUE 4: DATOS DE ENTRADA Y EJECUCION DEL CASO DE ESTUDIO
4 # -----
5
6 print("")
7 print("PROBLEMA 4.a: ")
8 print("")
9
10 print("ARISTAS: ")
11 aristas = {'O-A': 2, 'O-B': 5, 'O-C': 4, 'C-B': 1, 'A-B': 2, 'A-D': 8,
12           'B-D': 4, 'C-D': 3, 'E-D': 1, 'C-E': 4}
13 print(aristas)
14 print("")
15
16 print("NODOS: ")
17 nodos = ['O', 'A', 'B', 'C', 'D', 'E']
18 print(nodos)
19 print("")
20
21 print("NODO RAIZ: ")
22 nodo_raiz = 'O'
23 print(nodo_raiz)
24 print("")
25
26 print("SOLUCION CON DIJKSTRA: ")
27 print (Algoritmo_Dijkstra_final(aristas, nodos, nodo_raiz))
```

Listing 2: Datos de Entrada y Ejecución para el Problema 4.a

3. ALGORITMO DE PRIM

El **Algoritmo de Prim** es un algoritmo voraz diseñado para encontrar el **Árbol de Expansión Mínima (AEM)** de un grafo conexo, no dirigido y con pesos en sus aristas. A diferencia de Dijkstra, Prim se centra en conectar todos los nodos utilizando la mínima suma total de pesos en las aristas, construyendo el árbol de forma incremental.

- ✓ Su requisito principal es que el grafo sea **conexo**. Funciona correctamente con pesos **positivos** (o no negativos), e incluso con pesos negativos, siempre que el grafo no esté desconectado.
- ✓ El algoritmo exhibe una complejidad de tipo polinomial.
- ✓ Su mecánica se basa en mantener dos conjuntos de nodos: C para nodos **conectados** (ya incluidos en el árbol) y $N \setminus C$ para nodos **no conectados** (fuera del árbol).
- ✓ La selección se guía por el criterio de **la arista de menor peso** que conecta un nodo en C con uno en $N \setminus C$.

3.1. Fases del Procedimiento Iterativo para Prim

PASO 0. INICIALIZACIÓN

Se elige un nodo arbitrario (o raíz) r .

$$C = \{r\} \quad \text{y} \quad N \setminus C = N - \{r\}$$

$AEM = \emptyset$ (El conjunto de aristas del Árbol de Expansión Mínima está vacío).

PASO 1. SELECCIONAR LA ARISTA DE MÍNIMO COSTE

1. Elegir la arista (i, j) de menor peso que conecte un nodo i del conjunto **Conectado** (C) con un nodo j del conjunto **No Conectado** ($N \setminus C$):

$$(i^*, j^*) = \arg \min \{d_{ij} \mid i \in C, j \in N \setminus C\}$$

Donde i^* es el nodo de origen dentro del AEM actual y j^* es el nuevo nodo a añadir.

2. **Condición de Parada:** Si el conjunto de nodos no conectados ($N \setminus C$) está vacío, o si no existe una arista accesible (mínimo es ∞), **PARAR** el algoritmo.

PASO 2. ACTUALIZACIÓN DEL ÁRBOL

1. Añadir la arista seleccionada (i^*, j^*) al conjunto del AEM:

$$AEM \leftarrow AEM \cup \{(i^*, j^*)\}$$

2. Mover el nodo j^* (el nuevo nodo conectado) al conjunto **Conectado** (C):

$$C \leftarrow C \cup \{j^*\} \quad \text{y} \quad N \setminus C \leftarrow N \setminus C - \{j^*\}$$

3. Volver al **PASO 1**.

3.2. Implementación del Algoritmo de Prim en Python

Para la implementación del **Algoritmo de Prim** utiliza Python cuyo código se encuentra en el archivo `Algoritmo_Prim.py` basandose en la representación del grafo mediante una matriz de adyacencia y un enfoque greedy para construir el Árbol de Expansión Mínima (AEM) a partir de un nodo raíz, seleccionando iterativamente la arista de menor peso que conecta el conjunto de nodos ya incluidos (C) con un nodo fuera de él ($N \setminus C$).

El código se estructura en tres bloques principales: la construcción del grafo, el algoritmo principal y la aplicación a datos concretos.

```
1 INF = float('inf')
2
3
4 # -----
5 # BLOQUE 1: REPRESENTACION DEL GRAFO POR MATRIZ DE ADYACENCIA
6 # -----
7
8 def aristas_a_matriz(aristas: dict, nodos: list):
9
10     """
11     Convierte la representaciOn del grafo de aristas (diccionario) a una
12     matriz de adyacencia. ASUME UN GRAFO NO DIRIGIDO .
13     """
14
15     num_nodos = len(nodos)
16
17     # 1. Inicializar la matriz con 0 en la diagonal e INF en el resto
18     matriz_adyacencia = []
19     for i in range (0,num_nodos):
20         fila = []
21         for j in range (0,num_nodos):
22             if (i==j):
23                 fila.append(0.0)
24             else:
25                 fila.append(INF)
26         matriz_adyacencia.append(fila)
27
28     # 2. Mapear nombres de nodos a indices (0, 1, 2, ...)
29     mapeo_nodos = {nodo: i for i, nodo in enumerate(nodos)}
30
31     # 3. Rellenar la matriz con los pesos de las aristas
32     for arista, peso in aristas.items():
33         # Las aristas se esperan en formato 'Origen-Destino'
34         salida, llegada = arista.split('-')
35
36         i = mapeo_nodos[salida]
37         j = mapeo_nodos[llegada]
38         peso_float = float(peso)
39
40         # Asignacion directa (Salida -> Llegada)
41         matriz_adyacencia[i][j] = peso_float # Por simetria
42
43         matriz_adyacencia[j][i] = peso_float
44
45     return matriz_adyacencia
```

```

45 # -----
46 # BLOQUE 2: ALGORITMO PRINCIPAL DE PRIM
47 # -----
48
49 def Algoritmo_Prim(aristas: dict, nodos: list, nodo_raiz):
50
51     """
52     Implementacion del Algoritmo de Prim.
53     """
54
55     num_nodos = len(nodos)
56     mapeo_nodos = {nodo: i for i, nodo in enumerate(nodos)}
57
58     # 1. Validacion de la raiz
59     if nodo_raiz not in mapeo_nodos:
60         return "ERROR: Nodo raiz no encontrado.", None, None
61
62     matriz_adyacencia = aristas_a_matriz(aristas, nodos)
63
64     C = {nodo_raiz} # Conjunto de nodos ya incluidos en el arbol
65     grafo = {}
66     m = 1 # Contador de nodos en el conjunto C
67     peso = 0 # Peso total del AEM
68
69     # El bucle se ejecuta hasta que todos los nodos esten en C
70     while m < num_nodos:
71         nuevo_nodo = ''
72         minimo = INF
73         arista = ''
74         origen = ''
75
76         # a. Buscar arista mas barata que una C con nodo externo.
77         for i in C :
78             for j in nodos :
79                 if j not in C :
80                     arista_candidata = matriz_adyacencia[mapeo_nodos[i]
81 ][mapeo_nodos[j]]
82                     if (arista_candidata < minimo and arista_candidata
83 != INF ):
84                         minimo = arista_candidata
85                         nuevo_nodo = j
86                         origen = i
87
88         # b. Manejo de desconexion
89         if nuevo_nodo == '' :
90             return 'INFACTIBLE', None, None
91
92         # c. Incluir el nuevo nodo y la arista al arbol
93         C.add(nuevo_nodo)
94         arista = origen + '-' + nuevo_nodo
95         grafo[arista] = minimo
96         m = m + 1
97         peso += minimo
98
99     return 'EXITO', grafo, peso

```

Listing 3: Código Python del Algoritmo de Prim (Algoritmo.Prim.py)

```

1
2 # -----
3 # BLOQUE 3: DATOS DE ENTRADA Y EJECUCION DEL CASO DE ESTUDIO
4 # -----
5
6 # Conjunto de datos para el Problema 4.b
7 nodos_fuenfria = ['O', 'A', 'B', 'C', 'D', 'E']
8
9 # Los datos de entrada del ejercicio
10 aristas_fuenfria = {
11     'O-A': 2, 'O-B': 5, 'O-C': 4,
12     'C-B': 1, 'A-B': 2, 'A-D': 8,
13     'B-D': 4, 'C-D': 3, 'E-D': 1,
14     'C-E': 4
15 }
16
17 print("")
18 print("PROBLEMA 4.b: ")
19 print("")
20
21 print("ARISTAS: ")
22 print(aristas_fuenfria)
23 print("")
24
25 print("NODOS: ")
26 print(nodos_fuenfria)
27 print("")
28
29 print("NODO RAIZ: ")
30 nodo_raiz = 'O'
31 print(nodo_raiz)
32 print("")
33
34 print("SOLUCION CON PRIM: ")
35 # Ejecucion del Algoritmo de Prim
36 solucion_prim = Algoritmo_Prim(aristas_fuenfria, nodos_fuenfria, 'O')
37 print(solucion_prim)

```

Listing 4: Datos de Entrada y Ejecución para el Problema 4.b

4. APLICACIÓN AL PROBLEMA DE FUENFRÍA

Para la aplicación práctica de los algoritmos de Dijkstra y Prim, se ha seleccionado el Ejercicio 4 de la biblioteca de problemas de la sección Optimización en redes de los apuntes generales de la asignatura. El contexto se centra en la gestión de una red de transporte con minibuses en el parque de la Fuenfría.

Problema 4: *La Agencia del Medioambiente ha decidido limitar el acceso al parque de la Fuenfría. No se permitirán vehículos particulares y se utilizarán únicamente minibuses conducidos por guardias forestales. Las carreteras existentes (de un único sentido, siendo **O** la estación de entrada y A, B, C, D y E los lugares de interés) y sus distancias son las siguientes:*

$$\begin{aligned} O-A &= 2, O-B = 5, O-C = 4, C-B = 1, A-B = 2, \\ A-D &= 8, B-D = 4, C-D = 3, E-D = 1, C-E = 4. \end{aligned}$$

4.1. Parte a) Cálculo de Caminos Más Cortos

En el apartado a) del Problema 4 se pide: encontrar los caminos más cortos de la entrada al resto de las estaciones.

Para hallar los **caminos más cortos** desde la estación de entrada (**O**) al resto de los puntos de interés, se aplica el **Algoritmo de Dijkstra**. Esta elección se justifica por la necesidad de encontrar el camino mínimo desde un único origen a todos los destinos y por cumplir con el requisito de que todas las aristas tienen distancias no negativas. El problema se modela como un **grafo dirigido**, cuya estructura con las distancias correspondientes se ilustra en la Figura 1.

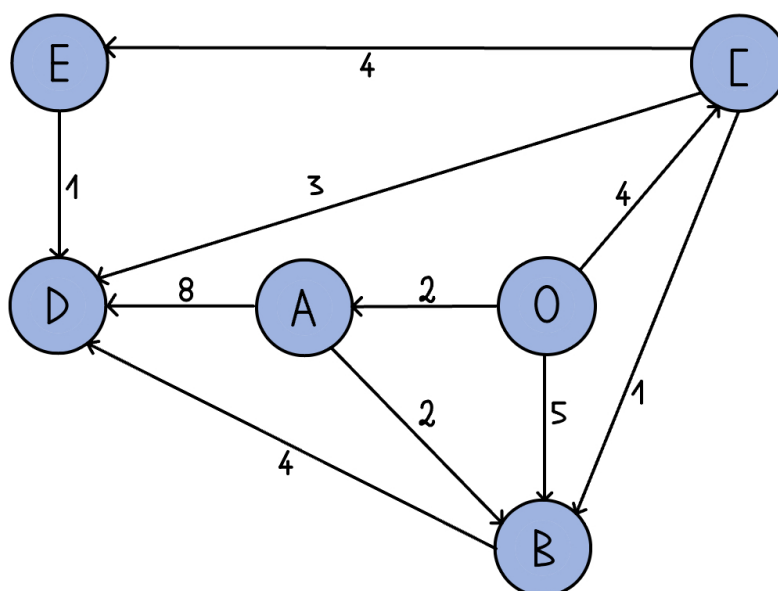


Figura 1: Representación del grafo dirigido con las distancias de la red de Fuenfría.

$$M = \begin{pmatrix} & \text{O} & \text{A} & \text{B} & \text{C} & \text{D} & \text{E} \\ \text{O} & 0 & 2 & 5 & 4 & \infty & \infty \\ \text{A} & \infty & 0 & 2 & \infty & 8 & \infty \\ \text{B} & \infty & \infty & 0 & \infty & 4 & \infty \\ \text{C} & \infty & \infty & 1 & 0 & 3 & 4 \\ \text{D} & \infty & \infty & \infty & \infty & 0 & \infty \\ \text{E} & \infty & \infty & \infty & \infty & 1 & 0 \end{pmatrix}$$

Figura 2: Matriz de Adyacencia para el Problema 4.a.

Nota: Las filas representan los nodos de origen y las columnas representan los nodos de destino.

4.1.1. Análisis de la Solución Obtenida

La ejecución del **algoritmo de Dijkstra** sobre los datos de la red arroja el siguiente conjunto de caminos más cortos y distancias mínimas:

```
PROBLEMA 4.a:

ARISTAS:
{'O-A': 2, 'O-B': 5, 'O-C': 4, 'C-B': 1, 'A-B': 2, 'A-D': 8, 'B-D': 4, 'C-D': 3, 'E-D': 1, 'C-E': 4}

NODOS:
['O', 'A', 'B', 'C', 'D', 'E']

NODO RAÍZ:
O

SOLUCIÓN CON DIJKSTRA:
{'Estado': 'Éxito Total', 'Grafo_Orientado': ['O-C', 'C-E', 'O-A', 'C-D', 'A-B'], 'Distancias': {'O': 0.0, 'A': 2.0, 'B': 4.0, 'C': 4.0, 'D': 7.0, 'E': 8.0}, 'Detalle': 'El grafo final contiene solo las aristas que forman los caminos más cortos desde la raíz.'}
```

Figura 3: Ejecución en Python del problema 4.a.

A continuación se detallan los resultados:

- La distancia más corta a **A** es de **2 km** (Camino: O → A).
- La distancia más corta a **B** es de **4 km** (Camino: O → A → B).
- La distancia más corta a **C** es de **4 km** (Camino: O → C).
- La distancia más corta a **D** es de **7 km** (Camino: O → C → D).
- La distancia más corta a **E** es de **8 km** (Camino: O → C → E).

Resultados Completos y Caminos Mínimos:

Destino	Distancia Mínima (km)	Camino Óptimo
O	0	O
A	2	O → A
B	4	O → A → B
C	4	O → C
D	7	O → C → D
E	8	O → C → E

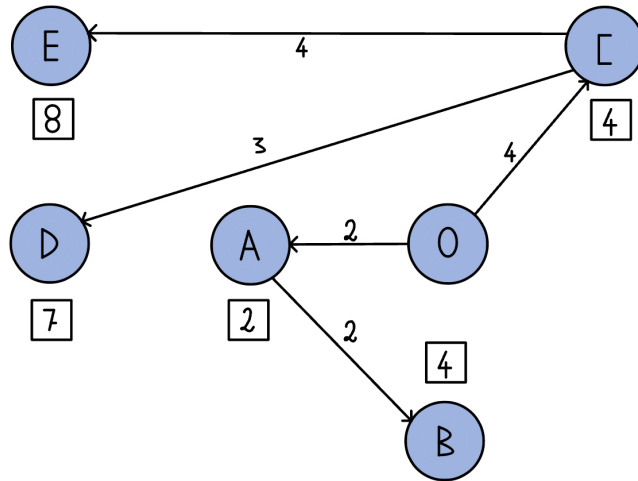


Figura 4: Grafo resultante con los Caminos Más Cortos desde O.

Conclusión: El Algoritmo de Dijkstra identificó las rutas de menor distancia a todas las estaciones desde el punto de origen O. La solución revela que, en varios casos, los caminos óptimos no son los directos, sino aquellos que pasan por un nodo intermedio, como es el caso de B, cuya distancia más corta se logra a través de A. El tiempo total mínimo de distribución está determinado por la estación más lejana, E, a 8 km.

4.2. Parte b) Minimizar Kilómetros de Línea Tendida

En el apartado b) se pide: si se desea comunicar con terminales de ordenador las estaciones, tendiendo líneas que sigan la carretera, resolver el problema de minimizar el número de kilómetros de línea tendida.

El problema de **minimizar la longitud** total de línea tendida equivale a encontrar el **Árbol de Expansión Mínima (AEM)** del grafo, resuelto mediante el Algoritmo de Prim. Dado que el tendido físico de la línea utiliza la carretera como ruta bidireccional, el grafo se modela como un **grafo no dirigido** representado en la Figura 5.

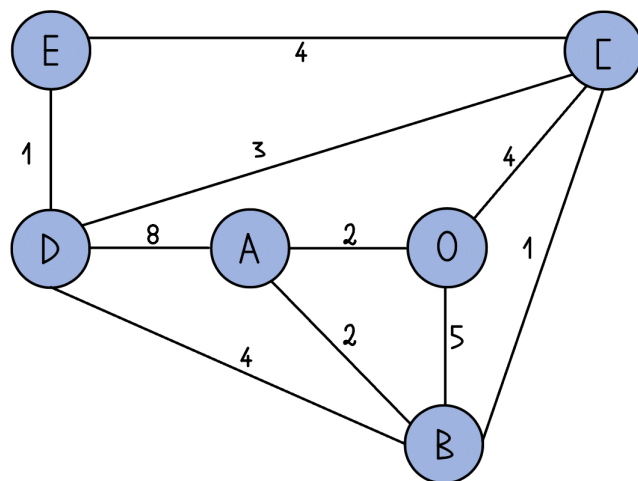


Figura 5: Representación del grafo con las distancias de la red de Fuenfría.

$$M = \begin{pmatrix} & \text{O} & \text{A} & \text{B} & \text{C} & \text{D} & \text{E} \\ \text{O} & 0 & 2 & 5 & 4 & \infty & \infty \\ \text{A} & 2 & 0 & 2 & \infty & 8 & \infty \\ \text{B} & 5 & 2 & 0 & 1 & 4 & \infty \\ \text{C} & 4 & \infty & 1 & 0 & 3 & 4 \\ \text{D} & \infty & 8 & 4 & 3 & 0 & 1 \\ \text{E} & \infty & \infty & \infty & 4 & 1 & 0 \end{pmatrix}$$

Figura 6: Matriz de Adyacencia para el Problema 4.b.

Nota: Las filas representan los nodos de origen y las columnas los nodos de destino.

4.2.1. Análisis de la Solución Obtenida

La ejecución del **Algoritmo de Prim** sobre los datos de la red arroja el **Árbol de Expansión Mínima (AEM)**, que es el conjunto de aristas necesario para conectar todos los nodos con la **mínima longitud total** de línea tendida:

```
PROBLEMA 4.b:

ARISTAS:
{'O-A': 2, 'O-B': 5, 'O-C': 4, 'C-B': 1, 'A-B': 2, 'A-D': 8, 'B-D': 4, 'C-D': 3, 'E-D': 1, 'C-E': 4}

NODOS:
['O', 'A', 'B', 'C', 'D', 'E']

NODO RAÍZ:
O

SOLUCIÓN CON PRIM:
('EXITO', {'O-A': 2.0, 'A-B': 2.0, 'B-C': 1.0, 'C-D': 3.0, 'D-E': 1.0}, 9.0)
```

Figura 7: Ejecución en Python del problema 4.b.

El AEM está compuesto por las siguientes aristas (distancias):

- La arista seleccionada para conectar **O** y **A** es de **2 km** (Arista: O - A).
- La arista seleccionada para conectar **A** y **B** es de **2 km** (Arista: A - B).
- La arista seleccionada para conectar **B** y **C** es de **1 km** (Arista: B - C).
- La arista seleccionada para conectar **C** y **D** es de **3 km** (Arista: C - D).
- La arista seleccionada para conectar **D** y **E** es de **1 km** (Arista: D - E).

Resultado Final del Árbol de Expansión Mínima (AEM):

Paso	Arista Seleccionada	Peso Acumulado (km)
1	O - A	2 = (0+2)
2	A - B	4 = (2+2)
3	B - C	5 = (4+1)
4	C - D	8 = (5+3)
5	D - E	9 = (8+1)
Costo Total Mínimo (AEM)		9 km

La **longitud total mínima** de línea requerida para comunicar todas las estaciones es de **9 km**.

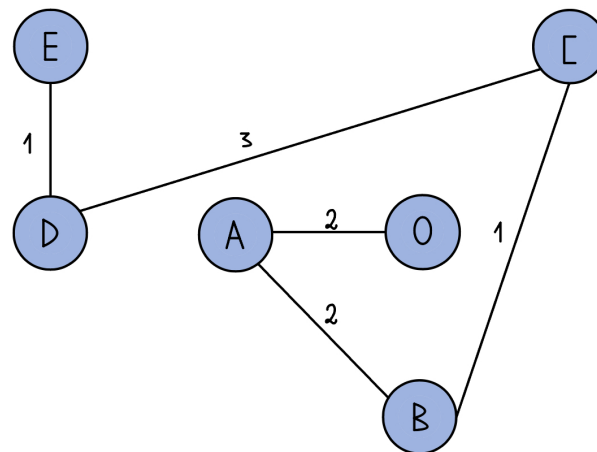


Figura 8: Grafo resultante con las aristas que componen el Árbol de Expansión Mínima.

Conclusión: El Algoritmo de Prim determinó que la **longitud total mínima** de línea requerida para garantizar que todas las estaciones estén comunicadas es de **9 km**. Este resultado se logró mediante una estrategia voraz que, en cada paso, añadió la arista de menor coste que conectaba el árbol en crecimiento con un nodo externo, cumpliendo el objetivo de minimizar los kilómetros totales de tendido de línea. Las aristas críticas seleccionadas son (O-A), (A-B), (B-C), (C-D), y (D-E).